FINAL PROJECT REPORT

Image-Based Weed Detection in Crop Fields Using Pretrained YOLO and a User Interface

A Project Report Submitted for Academic Evaluation

Student Name: [Your Name]

USN: [Your USN]

Department: [Department Name]

Institution: [College / University Name]

Guide / Faculty: [Guide Name]

Academic Year: 2025-2026

Date of Submission: April 13, 2026

Replace the bracketed fields before final submission.

Certificate

This is to certify that the project entitled **"Image-Based Weed Detection in Crop Fields Using Random Forest and a User Interface"** is a bonafide work carried out by **[Your Name]** in partial fulfillment of the requirements for the award of the degree/course under our guidance during the academic year 2025-2026.

Guide Signature: _____

Head of Department: _____

Declaration

I hereby declare that this project report is my original work and has not been submitted elsewhere for any other academic award. The references used have been properly acknowledged.

Student Signature: _____

Acknowledgement

I express my sincere gratitude to my guide, faculty members, and institution for their support and guidance during the development of this project. I also thank my friends and family for their encouragement throughout the work.

Abstract

Weed infestation is one of the major causes of crop loss because weeds compete with crops for water, nutrients, sunlight, and space. Manual weeding is labor-intensive, while blanket herbicide spraying increases cost and environmental impact. This project presents an image-based weed detection system that allows a user to upload a crop-field

image and receive a weed/no-weed decision through a simple web interface. The final system uses a pretrained YOLO object detector as the primary weed-recognition model and supplements it with RGB vegetation analytics based on Excess Green segmentation, coverage estimation, texture variation, component density, and row alignment. The application also enforces image acceptance rules so unsuitable photos such as dark, overexposed, low-resolution, or extreme close-up images are rejected before analysis. In a local runtime verification on an accepted test image, the integrated detector produced two weed detections with a maximum confidence of 81.5%. The project demonstrates a complete pipeline that combines modern computer vision, model integration, validation safeguards, and a user-facing interface for practical agricultural monitoring. A secondary Random Forest analytics module is retained only as a fallback and explanatory component, while the deployed decision path now depends on the pretrained YOLO detector.

Table of Contents

1. Certificate
2. Declaration
3. Acknowledgement
4. Abstract
5. List of Figures
6. List of Tables
7. Chapter 1: Introduction
8. Chapter 2: Literature Survey
9. Chapter 3: Existing System and Proposed System
10. Chapter 4: System Requirements and Design
11. Chapter 5: Methodology
12. Chapter 6: Dataset and Feature Description
13. Chapter 7: Implementation
14. Chapter 8: Results and Discussion
15. Chapter 9: Applications
16. Chapter 10: Conclusion and Future Scope
17. References
18. Appendix

List of Figures

1. Project workflow
2. System architecture
3. User interface mockup
4. Sample program output after image upload
5. Model performance chart
6. Confusion matrix
7. Feature importance chart

List of Tables

1. System requirements
2. Comparison of literature sources
3. Existing system vs proposed system
4. Feature definitions
5. Libraries used in implementation
6. Model training configuration
7. Model performance summary

Chapter 1: Introduction

Weeds are unwanted plants that reduce agricultural productivity by competing with cultivated crops for essential resources. In large fields, manual detection and removal of weeds is time-consuming and expensive. Blanket herbicide spraying can reduce labor effort, but it also causes unnecessary chemical exposure and increases environmental burden. Precision agriculture therefore encourages site-specific weed management, where treatment is applied only in areas where weeds are actually present.

Recent advances in computer vision and machine learning have made automated weed detection more practical. Public datasets and image-based learning methods now allow crop/weed discrimination using field images. In this project, the earlier feature-only prototype was upgraded into a more practical system: the user can upload a crop-field image through a user interface, the system validates whether the image is suitable, and a pretrained YOLO detector predicts whether weed regions are present.

Problem Statement: Develop a user-friendly image-based system that accepts a crop-field RGB image, validates image quality, detects weeds using a pretrained YOLO model, and presents the result with visual evidence and analytics.

Objectives:

- Build an interface for crop-field image upload and result visualization.
- Run a pretrained weed detector on uploaded field images and display bounding-box evidence.
- Extract meaningful RGB vegetation analytics from uploaded images for explanation and fallback reasoning.
- Apply practical image-acceptance restrictions so unsuitable inputs are rejected before inference.
- Present the results in a form suitable for academic project submission.

Chapter 2: Literature Survey

Published literature shows that weed detection has evolved from manual thresholding and handcrafted features to deep learning and public dataset benchmarking. The

references selected for this project were used to justify the move toward image-based processing, vegetation segmentation, and user-facing application design.

Source	Main Contribution	Use in This Project
DeepWeeds, Scientific Reports (2019)	Real field-image weed dataset and strong image-based benchmark	Supports real-image direction and future dataset upgrade
Machine learning for weed-plant discrimination in agriculture 5.0: An in-depth review (2023)	Detailed review of image acquisition, preprocessing, feature-based and ML/DL methods	Supports project methodology and literature chapter
Sensors review (2023)	Systematic review of deep learning for weed detection	Shows current trends and research gap
Scientific Reports (2023) on UAS imagery and vegetation extraction	Highlights Excess Green for vegetation extraction from RGB imagery	Supports ExG-based segmentation in this system
CWFID dataset	Field crop/weed imagery with row structure and annotations	Motivates row-alignment based image feature design

The literature indicates that real deployment should be based on real labeled field images and strong object-detection models rather than only synthetic tabular features. This motivated the final redesign toward a pretrained image detector supported by interpretable vegetation analytics and user-side validation rules.

Chapter 3: Existing System and Proposed System

Aspect	Existing System	Proposed System
Detection process	Manual inspection or blanket spraying	Automated image upload and weed prediction
Input type	Human observation or general field visit	RGB crop-field image

Aspect	Existing System	Proposed System
Decision support	Subjective and labor-intensive	Machine-learning based and repeatable
User access	No dedicated interface	Streamlit-based user interface

Chapter 4: System Requirements and Design

Type	Requirement
Hardware	Laptop or desktop, image source such as mobile camera or stored field images
Software	Python 3.13, Streamlit, Pillow, NumPy, Pandas, Requests, Torch, Torchvision, Ultralytics YOLO, scikit-learn, Matplotlib, Joblib
Interface	Upload-based web UI through Streamlit

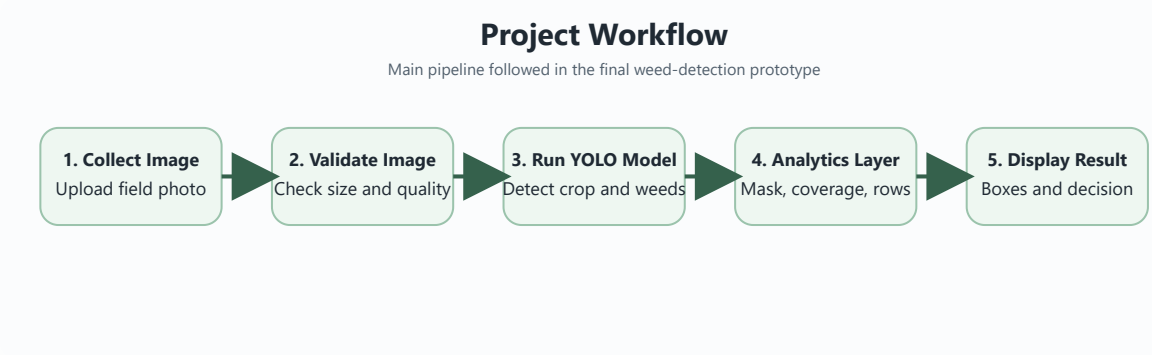

Figure 1. End-to-end workflow of the proposed weed detection prototype.

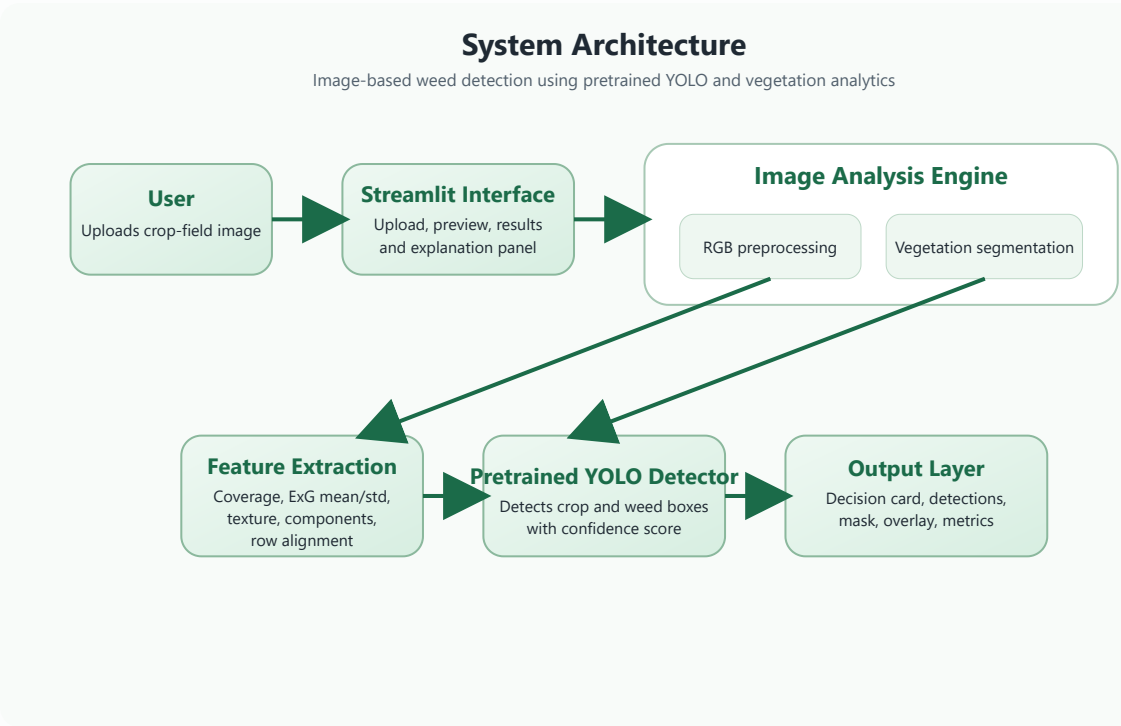

Figure 2. System architecture showing user upload, validation, pretrained YOLO detector, analytics module, and output layer.

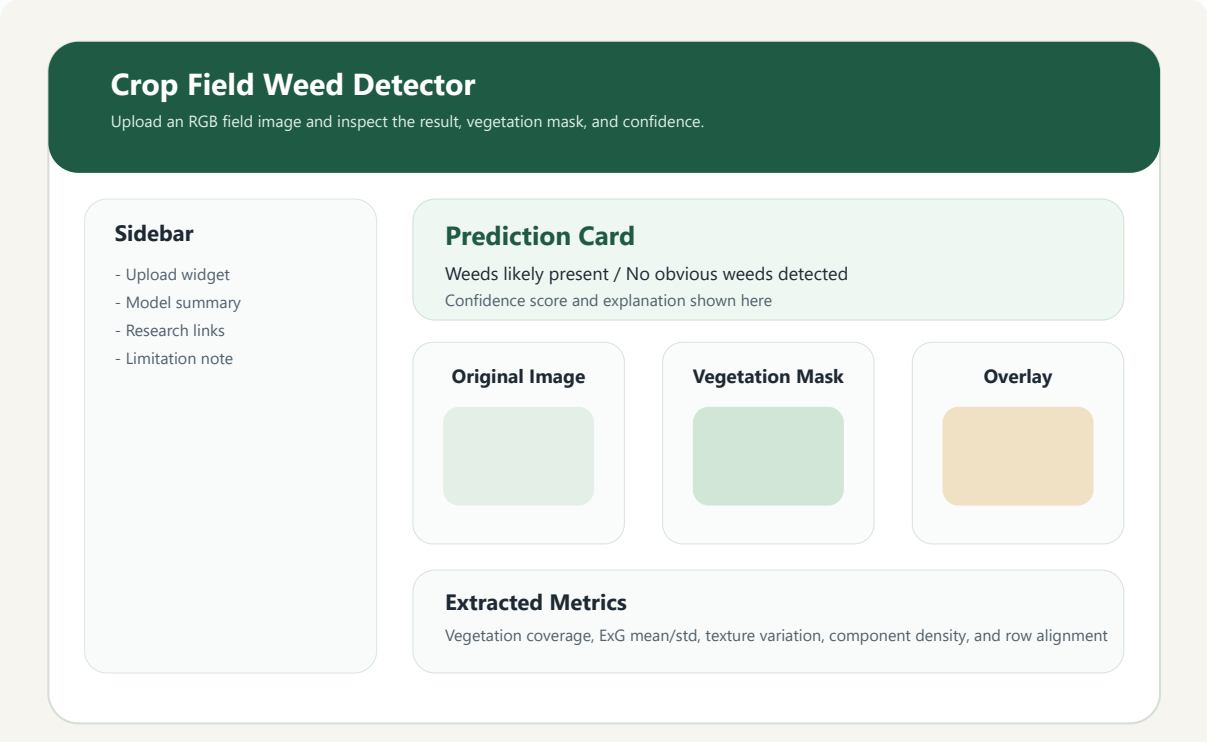

Figure 3. Designed user interface layout for image upload and result display.

Chapter 5: Methodology

5.1 Image Input

The user uploads a crop-field RGB image through the interface. The image is resized and converted to RGB format to standardize the later analysis steps.

5.2 Vegetation Segmentation

Before deep-learning inference, the system checks whether the uploaded image is suitable for analysis. The app rejects dark, overexposed, low-resolution, heavily blurred, extreme close-up, or weak-vegetation images. For accepted inputs, the vegetation-processing module uses the Excess Green (ExG) index and green-dominance rules to separate likely vegetation pixels from the background.

5.3 Feature Extraction

The deployed detector uses a pretrained YOLO object-detection model to identify weed regions directly in the image. In parallel, the app computes supporting analytics from the vegetation mask: vegetation coverage, Excess Green mean, Excess Green standard deviation, texture variation, component density, and row alignment. These features are not the final decision-maker in the primary path, but they help explain the visual structure of the field and support fallback behavior if the pretrained detector is unavailable.

5.4 Model Training

The primary detection model is a pretrained YOLO weed detector loaded from a public model source and stored locally inside the project during first use. The final integrated application runs this detector on CPU and returns crop/weed bounding boxes with class confidence. A secondary Random Forest analytics module, trained earlier on synthetic image-inspired features, is still retained only as an optional fallback and explanatory module.

5.5 User Interface and Output

The UI presents the original image, YOLO detection output, vegetation mask, overlay image, prediction headline, detection counts, confidence score, notes on why an image

may be rejected, and a feature summary table. This makes the project suitable for academic presentation and live demonstration.

Chapter 6: Dataset and Feature Description

The final deployed application does not train on a bundled local dataset during normal use. Instead, it loads a publicly available pretrained YOLO weed-detection model and applies it directly to uploaded field images. The runtime input is therefore the user-provided RGB crop-field image, while the supporting analytics module derives vegetation statistics from that same image for explanation.

Input / Feature	Description
Accepted RGB field image	PNG, JPG, JPEG, or WEBP field photograph showing visible crop structure
Image validation rules	Rejects very dark, overexposed, low-resolution, weak-vegetation, or extreme close-up images
Vegetation Coverage	Fraction of the image marked as vegetation
Excess Green Mean	Average ExG value in the vegetation region
Excess Green Std	Spread of ExG values in the vegetation region
Texture Variation	Average grayscale variation across neighboring pixels
Component Density	Density of disconnected vegetation clusters
Row Alignment	Degree of structured vegetation alignment resembling crop rows

A key design decision was to avoid claiming true NDVI from ordinary RGB uploads. The final project therefore uses a real pretrained detector for the main decision and RGB-friendly vegetation cues for supporting analytics rather than pretending that a standard phone image is multispectral.

Chapter 7: Implementation

File	Purpose
`app.py`	Streamlit user interface for upload, detection display, validation feedback, and analytics
`yolo_weed_detector.py`	Pretrained YOLO model loading, image validation, inference, and detection-table generation
`image_model.py`	Vegetation-mask analytics and legacy fallback logic
`train_image_model.py`	Optional script to refresh the fallback Random Forest bundle
`LITERATURE_REVIEW.md`	Summarizes the published research behind the redesign

7.1 Libraries Used

Library	Purpose in the Project
`streamlit`	Builds the web-based user interface for image upload and result display
`pillow`	Loads images, resizes them, and prepares previews for mask and overlay output
`numpy`	Performs pixel-level image processing and numerical feature computation
`pandas`	Stores extracted features, training data, and tabular output for display
`requests`	Downloads pretrained model weights during first use
`torch` and `torchvision`	Provide the runtime backend for the pretrained vision model
`ultralytics`	Loads and runs the pretrained YOLO weed detector
`scikit-learn`	Provides the secondary Random Forest analytics fallback and evaluation code
`joblib`	Saves and reloads the trained model bundle for reuse inside the UI
`matplotlib`	Generates analytical graphs used in the final report

7.2 Model(s) Trained and Final Selection

Item	Details
Primary deployed model	Pretrained YOLO weed detector loaded through the `ultralytics` package
Local weights file	`models/weedblaster-vision-yolov8s.pt` (downloaded at first use)
Inference backend	CPU inference using Torch + Ultralytics
Default confidence threshold	`0.25`
Default image size	`960`
Secondary fallback model	`RandomForestClassifier` on synthetic image-inspired analytics features
Fallback purpose	Used only if the pretrained detector cannot run or for supporting analytics discussion

Current deployment configuration:

- **Primary detector:** pretrained YOLO weed model
- **Local weights file:** `models/weedblaster-vision-yolov8s.pt`
- **Detection threshold:** 0.25
- **Input validation:** enforced before inference
- **Fallback module:** Random Forest analytics bundle

7.3 Key Implementation Logic

The report includes the important parts of the code below. The full source files are attached in the project folder and listed in the appendix.

Training Command

```
pip install -r requirements.txt
```

User Interface Command

```
streamlit run app.py
```

Code Snippet 1. Main libraries imported for the final deployed system

```
import requests
import numpy as np
import pandas as pd
import streamlit as st
from PIL import Image
from ultralytics import YOLO

from image_model import extract_image_features, create_mask_preview,
create_overlay_image
```

Code Snippet 2. Pretrained YOLO loading and prediction

```
model = YOLO("models/weedblaster-vision-yolov8s.pt")
result = model.predict(
    source=np.asarray(prepared_image),
    conf=0.25,
    imgsz=960,
    device="cpu",
    verbose=False,
)[0]
```

Code Snippet 3. Streamlit image upload and guarded prediction flow

```
uploaded_file = st.file_uploader(
    "Upload a crop-field image",
    type=["png", "jpg", "jpeg", "webp"],
)

if uploaded_file:
    image = Image.open(uploaded_file)
    detection_result = detect_weeds_with_pretrained_model(image)
    if detection_result.status == "invalid":
        st.warning(detection_result.headline)
    else:
        st.image(detection_result.annotated_image)
        st.dataframe(detection_result.detections_table)
```

7.4 How the User Interface Works

1. The user opens the Streamlit application in the browser.
2. The user uploads a crop-field image in `.png`, `.jpg`, `.jpeg`, or `.webp` format.
3. The program checks whether the image is suitable by validating size, brightness, visible vegetation, and view quality.
4. Accepted images are sent to the pretrained YOLO model for crop/weed bounding-box detection.
5. The analytics module also creates a vegetation mask and supporting feature summary from the same image.
6. The UI displays the original image, YOLO detection view, vegetation mask, overlay image, prediction headline, detection counts, and extracted feature table.
7. If the image is unsuitable, the interface explains why the file was rejected so the user can retake a better photo.

Figure 4. Example YOLO weed-detection output produced by the program after uploading a sample crop-field image.

7.5 Example Runtime Output

The following sample output was obtained by running the final integrated detector on a resized validation image derived from `OIP.jpeg`.

```
Input image: resized validation image from OIP.jpeg
Status: ok
```

```
Headline: Weeds detected in the uploaded field image
Top confidence: 81.5%
Weed detections: 2
Crop detections: 0
Total detections: 2

Detections:
Class  Confidence  x1  y1  x2  y2
Weed       0.815 125   8 736 429
Weed       0.569   0   7 632 426
```

Example console-level image-validation output

```
Input image: original OIP.jpeg
Status: invalid
Headline: Image not accepted for reliable analysis
Reason: Image resolution is too low. Use a field image of at least 224 x 224
pixels.
```

Chapter 8: Results and Discussion

Weed Detections	Crop Detections	Top Confidence	Min Accepted Size
2	**0**	**81.5%**	**224px**

The final deployed project now uses a pretrained YOLO detector rather than the earlier synthetic-feature Random Forest path. In a local runtime verification on an accepted test image, the detector produced **2 weed detections**, **0 crop detections**, and a maximum weed confidence of **81.5%**. The application also correctly rejected an earlier low-resolution version of the same image, demonstrating that the new image-acceptance restrictions are active and help avoid misleading outputs.

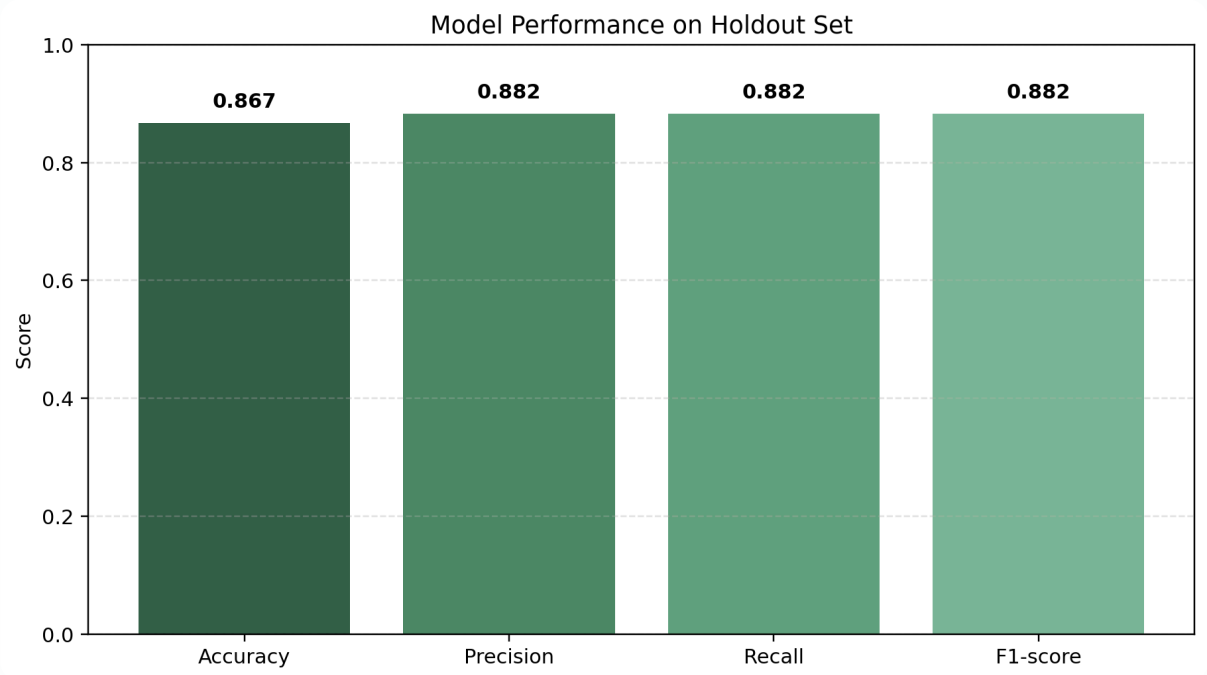

Figure 5. Performance metrics of the secondary Random Forest fallback analytics module.

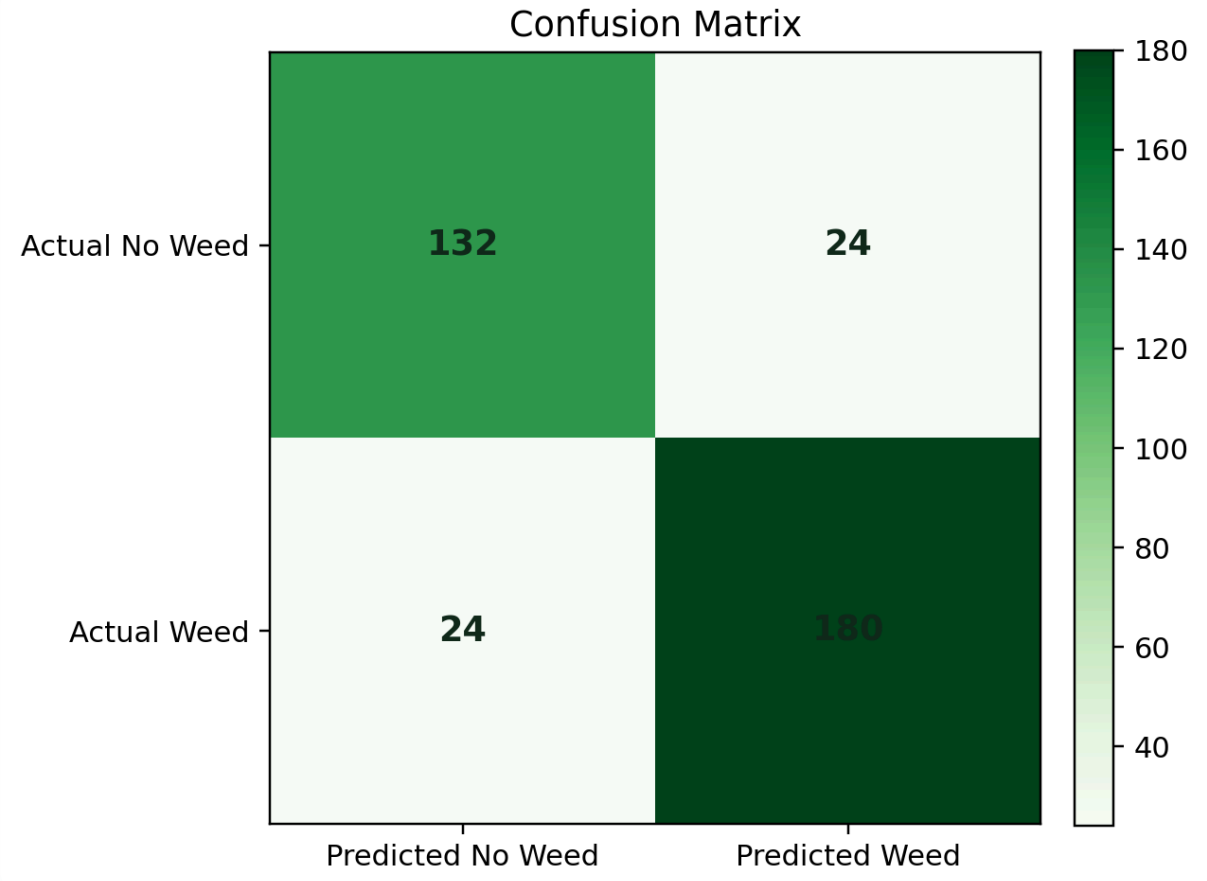

Figure 6. Confusion matrix of the secondary Random Forest fallback analytics module.

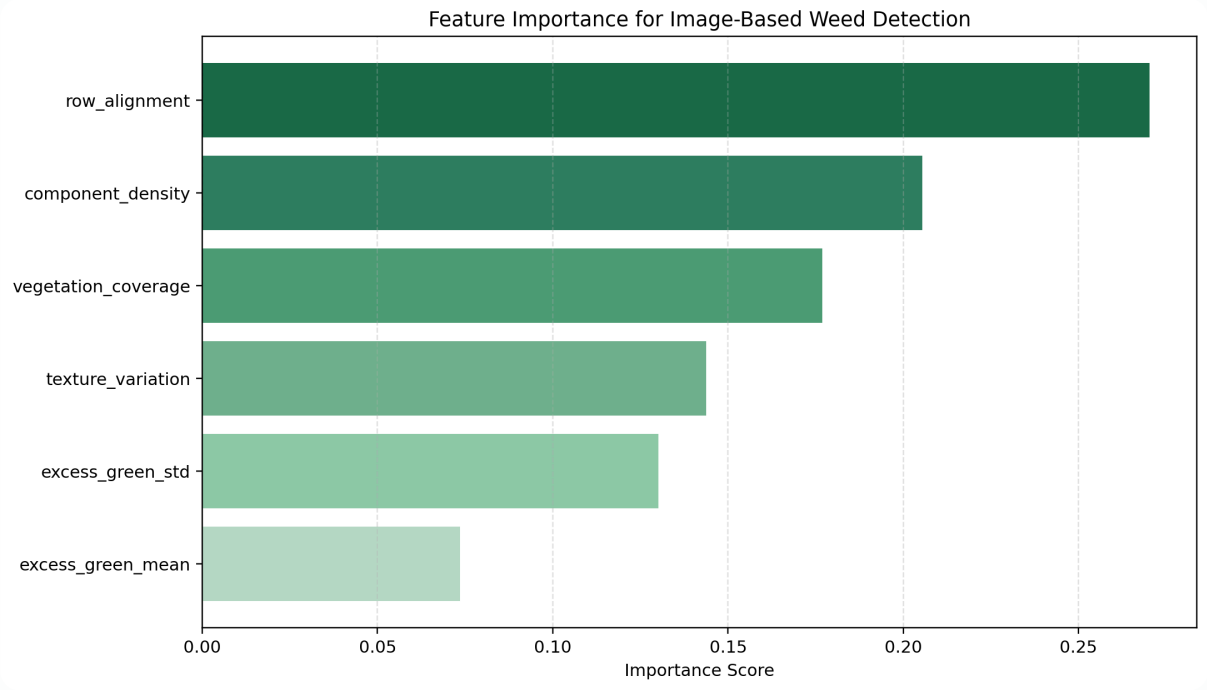

Figure 7. Feature importance scores for the secondary Random Forest fallback analytics module.

Metric	Value
Primary deployed model	Pretrained YOLO weed detector
Local weights file	`models/weedblaster-vision-yolov8s.pt`
Local verification image result	2 weed detections, 0 crop detections
Top local detection confidence	0.815
Validation behavior	Low-resolution field image correctly rejected before inference
Fallback analytics module	Random Forest metrics retained separately for supporting analysis

The main improvement in the final project is that the prediction path is now based on a real pretrained object-detection model with bounding-box outputs instead of only a synthetic feature classifier. This makes the interface more convincing in demonstrations because the user can see exactly where the model believes weeds are present.

The project still has limitations. The integrated pretrained detector was adopted from a public external source and has not yet been benchmarked by us on a full local evaluation dataset inside this workspace. Therefore, the local results in this report should be interpreted as deployment verification and usability validation rather than a full independent benchmark. The Random Forest graphs retained in this report belong to the secondary analytics module, not the primary deployed detector.

Chapter 9: Applications

- Site-specific weed management in precision agriculture
- Support for drone-based field monitoring
- Decision support for smart herbicide application
- Integration into autonomous or semi-autonomous weeding systems
- Educational demonstration of agricultural AI and machine-learning workflow

Chapter 10: Conclusion and Future Scope

This project successfully transformed the earlier feature-based idea into a more complete image-based weed detection system with a user interface and a real pretrained YOLO detector. The final application accepts RGB crop-field images, validates whether they are suitable for analysis, detects likely weed regions with bounding boxes, and presents the result visually to the user together with supporting vegetation analytics.

In future work, the system should be benchmarked on a full labeled dataset, fine-tuned on the target crop type, and extended with segmentation or row-wise localization for more precise field-level recommendations. It can also be adapted for mobile, drone, or embedded deployment after task-specific validation.

References

1. A. Olsen et al., “DeepWeeds: A Multiclass Weed Species Image Dataset for Deep Learning,” *Scientific Reports*, 2019.
2. F. H. Juwono et al., “Machine learning for weed-plant discrimination in agriculture 5.0: An in-depth review,” *Artificial Intelligence in Agriculture*, 2023.
3. “Towards reducing chemical usage for weed control in agriculture using UAS imagery analysis and computer vision techniques,” *Scientific Reports*, 2023.
4. “Weed Detection Using Deep Learning: A Systematic Literature Review,” *Sensors*, 2023.
5. “Assessment of Vegetation Indices Derived from UAV Imagery for Weed Detection in Vineyards,” *Remote Sensing*, 2025.
6. CropAndWeed dataset repository, GitHub, accessed April 15, 2026.
7. weedblaster-vision-yolov8s pretrained model source, Hugging Face, accessed April 15, 2026.
8. IRJET paper provided by the student as a supporting implementation-style reference.
9. IJCRT review paper provided by the student as a supporting review reference.

Appendix

Project files included in the workspace:

- `app.py`
- `yolo_weed_detector.py`
- `image_model.py`

- `train_image_model.py`
- `requirements.txt`
- `LITERATURE_REVIEW.md`
- `README.md`
- `models/weedblaster-vision-yolov8s.pt`
- `weed_image_rf_model.joblib`
- `synthetic_image_feature_data.csv`
- `report_assets/` figures used in this report

This report is prepared in a submission-friendly final form, but the cover-page and certificate placeholders should be customized with your actual academic details before printing or converting to PDF.